アジャイルワーク2　実験報告書草稿（確認用抜粋）

枝刈り・CSRによる
色補正モデルの圧縮

学籍番号　25G1017

氏名　岩澤大心

作成日　2026年11月13日

1　本稿の範囲

本稿は完成版から一部を省略した確認用草稿である．実施済みの実験に基づき，目的・理論・実験方法と，新規10回のMSE集計までを掲載する．

2　目的

Arduino上の色補正モデルについて，枝刈り＋CSRによる保存容量と推論時間の削減効果，および補正精度への影響を調べる．Denseに対するMSE増加5%以下を精度維持の条件とし，MSE300以下も別に判定する．

同じHのDenseと枝刈り＋CSRへ同一入力を与えて比較した．当初計画の40・50・60次元から，実施条件を40・60・80・100へ変更した．H50は未実施である．

3　比較の構成

図1のように，1回のセンサ読取から得たRGBを両方式へ入力した．4種類のHと2方式を組み合わせ，比較条件は8個とした．

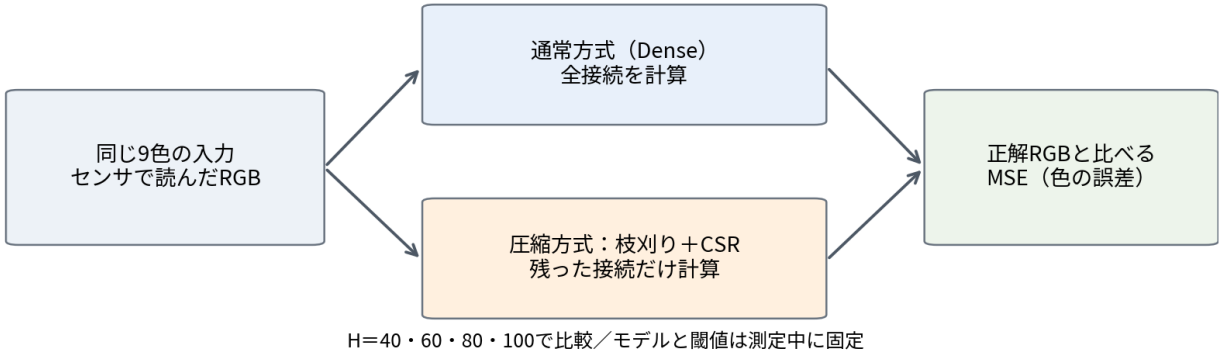


図1　同一入力による比較

用途	データ・操作
学習	過去の実測135行：3×3チャート111行，6×4チャート24行
閾値選択	別の8×8チャートを2回読んだ128入力と，学習内9入力
精度測定	学習にも使用した3×3チャートを新しく10回読取，計90入力
速度測定	保存済み実測9入力を実機で反復再生
メモリ比較	各モデル単独の8条件をコンパイル

新規10回の入力は学習・閾値選択へ加えていない．ただし，色自体は学習時と同じであり，独立した未見チャートによる最終評価ではない．

4　仕組みと計算方法

4.1　ネットワークと学習

センサの読取値を黒白校正で正規化し，3–H–H–3の全結合ネットワークへ入力した．図2は層の構成と，計算単位1個の処理を示す模式図である．Hは各隠れ層の計算単位数であり，入力・出力はRGBの3成分である．

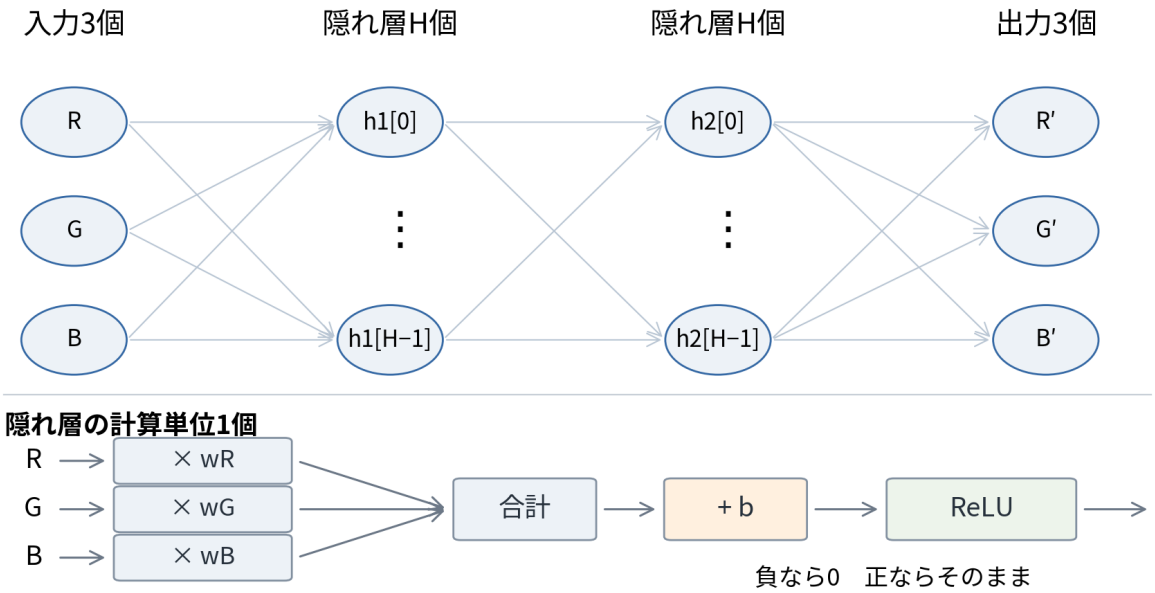


図2　ネットワークと計算単位

RGB成分cの読取値を v_c，黒・白の校正値を $v_{B,c}, v_{W,c}$ とすると，入力は式(1)で求める．clipは範囲外の値を0または1へ収める処理である．

$$x_c = \mathrm{clip}_{[0,1]}\left(\frac{v_c - v_{B,c}}{v_{W,c} - v_{B,c}}\right) \tag{1}$$

重みは接続ごとに入力へ掛ける係数，バイアスは各計算単位へ加える調整値である．最初の隠れ層の計算単位1個では，式(2)のように入力と重みの積を合計する．例えば入力(0.7,0.2,0.1)，重み(0.5,−0.2,0.3)，バイアス0.01なら，合計は0.35となる．これは仕組みの説明用の値である．

$$z = w_R x_R + w_G x_G + w_B x_B + b \tag{2}$$

全計算単位を行列でまとめたものが式(3)である．W_1, W_2, W_3 の大きさは順に $H \times 3, H \times H, 3 \times H$ となる．

$$\begin{aligned} h_1 &= \mathrm{ReLU}(W_1 x + b_1), \\ h_2 &= \mathrm{ReLU}(W_2 h_1 + b_2), \\ y &= W_3 h_2 + b_3 \end{aligned} \tag{3}$$

隠れ層には式(4)を用い，負の値を0にする．非線形処理を挟むことで，複数の線形変換だけでは表せない入力と出力の関係を学習できる．出力層は線形とし，評価時にクリップする．

$$\mathrm{ReLU}(z) = \max(0, z) \tag{4}$$

図3はPCでの学習の流れである．予測と正解から損失を計算し，`backward()`で勾配を求め，Adamで重み・バイアスを更新する．学習時は正解RGBを255で割った尺度を使用した．

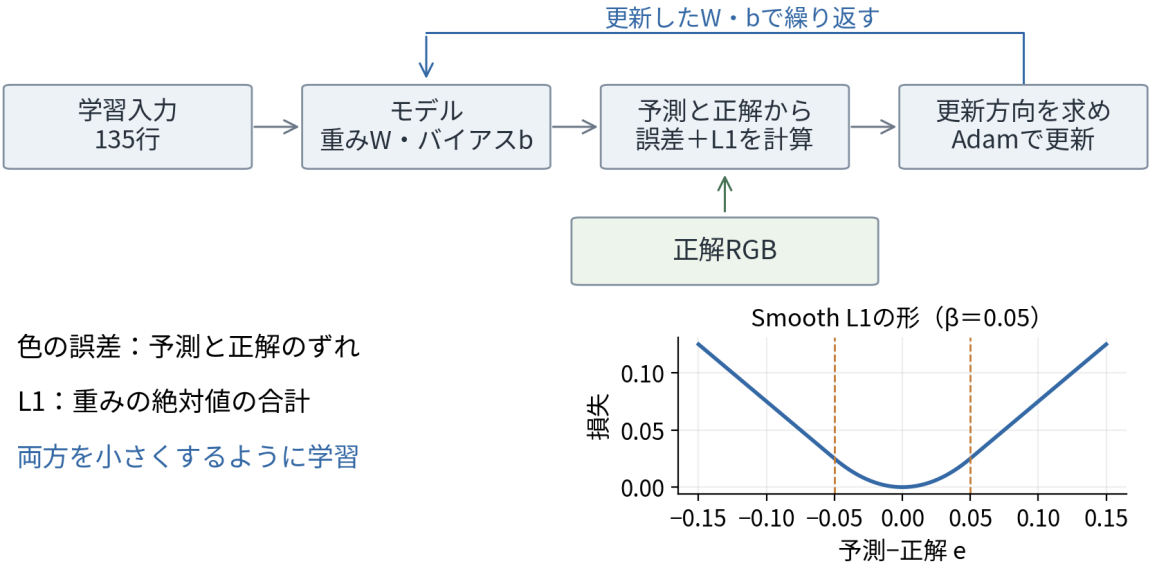


図3　学習と損失の関係

損失は式(5)とした．E_i は学習行iのRGB誤差の平均，$a_i > 0$ はその行の重要度，$M = 135$ は学習行数である．前半は行の重要度付き平均，後半はL1正則化[1]であり，重要度 a_i とモデルの重み W は区別する．

$$
\begin{aligned}
E_i &= \frac{1}{3} \sum_{c \in \{R,G,B\}} \rho_\beta(\hat{y}_{i,c} - t_{i,c}), \\
\mathcal{L} &= \frac{\sum_{i=1}^{M} a_i E_i}{\sum_{i=1}^{M} a_i} + \lambda \sum_{\ell=1}^{3} \sum_{j,k} |(W_\ell)_{jk}|, \qquad \lambda = 10^{-6}
\end{aligned}
\tag{5}
$$

RGB誤差には式(6)のSmooth L1[5]を用いた．e は予測と正解の差である．小さい誤差では二乗，大きい誤差では直線的に損失が増え，一部の大きな残差の影響を抑える．$\beta = 0.05$ はその切替値である．

$$
\rho_\beta(e) = \begin{cases} e^2/(2\beta) & (|e| < \beta), \\ |e| - \beta/2 & (|e| \geq \beta), \end{cases} \qquad \beta = 0.05 \tag{6}
$$

L1項は3層の重みの絶対値を足し，大きな重みへ罰則を与える．層番号を ℓ，行列内の位置をj,kとし，バイアスは罰則の対象から除いた．小さな重みを作る方向に作用するが，重みを実際に0へ変更する処理は次節の枝刈りで行う．

4.2　枝刈りとCSR

枝刈り[2]では，学習後の重みへ式(7)を適用する．τ は閾値であり，絶対値が小さい重みを0にする．バイアスは変更しない．

$$
w^{(\tau)} = \begin{cases} 0 & (|w| < \tau), \\ w & (|w| \geq \tau) \end{cases} \tag{7}
$$

図4は閾値0.05を用いた説明例である．0.03や−0.02は0になり，0.5や−0.2は残る．実験で採用した閾値は表3に示す．枝刈りの τ と，Smooth L1の β は用途が異なる．

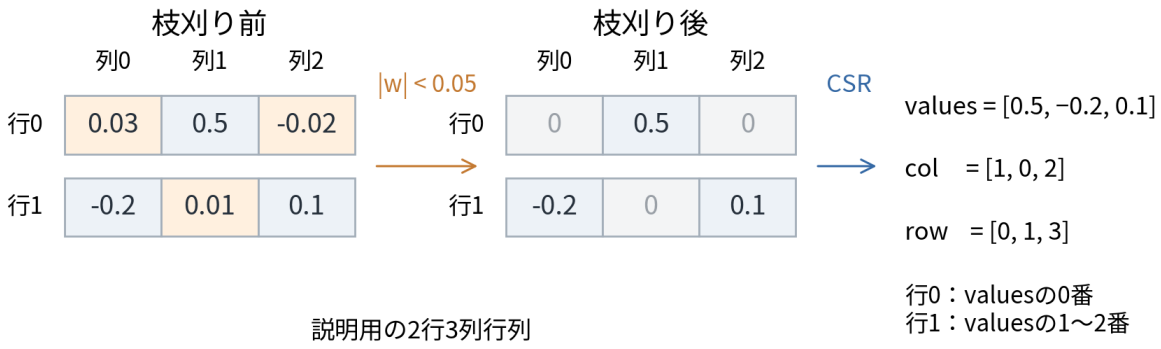


図4　枝刈りとCSR変換

0を配列に残したままでは保存領域が必要なため，非ゼロ重みだけをCSR[3]で保存する．`values` は重みの値，`col` は元の列番号，`row` は各行の開始位置と終端を表す．図4の `row=[0,1,3]` では，行0にvaluesの0番，行1に1〜2番を使う．列番号を使えば，省略した0の位置も含めて元と同じ積和計算ができる．

CSR変換は枝刈り後の行列を表現し直す処理であり，精度への影響は枝刈り前のDenseとの比較で確認する．3層の非ゼロ重み数をKとしたとき，式(8)は重みを0にした割合を表す．

$$S = 1 - \frac{K}{H^2 + 6H} \tag{8}$$

4.3　メモリ容量の手計算

図2の各接続に重みが1個ずつあるため，Denseの重み数は $N_w = 3H + H^2 + 3H$ となる．各計算単位のバイアスは $N_b = 2H + 3$ 個である．FP32は1個4 byteなので，式(9)で配列容量を求める．

$$B_{\text{Dense}} = 4(N_w + N_b) = 4(H^2 + 8H + 3) \quad [\text{byte}] \tag{9}$$

CSRは非ゼロ重みK個をFP32，列番号をuint8，行開始位置をuint16で保存する．容量はそれぞれ4K，K，$2(2H + 6)$ byteとなる．行位置の個数は各層の行数に終端を1個ずつ足した $(H + 1) + (H + 1) + (3 + 1) = 2H + 6$ 個である．バイアスを全て残すと，総容量は式(10)となる．

$$\begin{aligned} B_{\text{CSR}} &= 4K + K + 2(2H + 6) + 4(2H + 3) \\ &= 5K + 12H + 24 \quad [\text{byte}] \end{aligned} \tag{10}$$

H60では重み3,960個，バイアス123個，枝刈り後K=1,141個なので，式(11)のように計算できる．CSR行位置は61＋61＋4＝126個で252 byte，バイアスは123個で492 byteである．

$$\begin{aligned} B_{\text{Dense}} &= 4(60^2 + 8 \cdot 60 + 3) = 16,332, \\ B_{\text{CSR}} &= 4,564 + 1,141 + 252 + 492 = 6,449, \\ R_B &= 100(1 - 6,449/16,332) = 60.51\% \end{aligned} \tag{11}$$

全HのCSR配列の内訳を示す．単位はbyteである．

H	非ゼロ重み	列番号	行位置	バイアス	合計byte
40	1,880	470	172	332	2,854
60	4,564	1,141	252	492	6,449
80	6,596	1,649	332	652	9,229
100	9,496	2,374	412	812	13,094

CSRでは位置情報も保存するため，非ゼロ重みが多いと容量が減らない．$B_{\text{CSR}} < B_{\text{Dense}}$ を整理した式(12)を，採用した4条件は全て満たす．

$$5K < 4H^2 + 20H - 12 \tag{12}$$

この計算は数値配列のみを対象とし，構造体・アラインメント・Arduinoコア等を含むFlash全体はIDE表示で別に評価した．

4.4 評価指標

出力を0〜1へクリップし，255倍して四捨五入した整数RGBで評価した．9色の補正値 y と正解 t から，式(13)でMSEを求める．9色×RGBの3成分で，比較数は27個となる．

$$\text{MSE} = \frac{1}{27}\sum_{i=1}^{9}\sum_{c\in\{R,G,B\}}(y_{i,c} - t_{i,c})^2 \tag{13}$$

説明例として，1色の正解が(255,0,0)，予測が(245,10,5)で，残り8色が一致した場合，二乗誤差の合計は225，MSEは $225/27 = 8.33\ldots$ となる．MSEは小さいほど正解に近い．

各回のMSEを m_r とし，式(14)で10回平均と標本標準偏差（SD）を算出した．平均で誤差の大きさ，SDで読取ごとのばらつきを評価する．

$$\overline{m} = \frac{1}{10}\sum_{r=1}^{10} m_r, \qquad s_m = \sqrt{\frac{1}{9}\sum_{r=1}^{10}(m_r - \overline{m})^2} \tag{14}$$

MSE・標本SD・対応差は小数第2位まで表示し，第3位以降を切り捨てる．集計・増加率の計算には切り捨て前の値を使用する．

同じ読取回における差を式(15)で求める．正の値はCSRで誤差が増えたことを示す．

$$d_r = m_{r,\text{CSR}} - m_{r,\text{Dense}} \tag{15}$$

MSE増加率は式(16)で求め，5%以下を精度維持の条件とした．各回の割合を平均するのではなく，両方式の10回平均を比較する．

$$R_{\text{MSE}} = 100\left(\frac{\overline{m}_{\text{CSR}}}{\overline{m}_{\text{Dense}}} - 1\right) \quad [\%] \tag{16}$$

容量Bと推論時間Tの削減率は式(17)で算出した．いずれもDenseを基準とし，正の値が容量・時間の削減を表す．

$$R_B = 100\left(1 - \frac{B_{\text{CSR}}}{B_{\text{Dense}}}\right), \qquad R_T = 100\left(1 - \frac{T_{\text{CSR}}}{T_{\text{Dense}}}\right) \quad [\%] \tag{17}$$

5　実験方法

独立変数はHと方式，従属変数はMSE・モデル配列容量・Flash・静的RAM・推論時間とした．同じHでは，元モデル・入力・RGB丸め・ボード・計時条件をそろえた．H間では追加学習前の重みも異なる．

5.1　装置と実行環境

項目	実施条件
ボード	Arduino UNO R4 WiFi
主マイコン・クロック	Renesas RA4M1，48 MHz
公称Flash／SRAM	262,144／32,768 byte
センサ	A0へ接続した光センサ，型番未確認
LED	R:D8，G:D6，B:D7，analogWrite 255／0
読取設定	各チャンネル10サンプル平均，サンプル間2 ms，点灯後10 ms待機
操作	シリアル9600 bps，外付けD2/D3ボタンは不使用
PC環境	Mac，Jupyter Notebook
最終学習・集計	クラウドPython 3.12，PyTorch 2.8.0，NumPy 2.3.5
実機コンパイラ	7.2.1 20170904，ARM/embedded-7-branch revision 255204
Arduino IDE・コア	版番号未記録，コンパイラ文字列を保存

紙は手で支え，照明を同じ状態に保つ手順で測定した．照度・距離・角度の定量記録はない．

5.2　学習データと正解画像

学習データは実測135行（3×3分111行，6×4分24行）を使用した．3×3の教師値と評価には `result.ppm` を使用し，6×4分には配布Notebookの教師値を使用した．

各Hは同じHのV9重みから40,000 epoch追加学習した．seed42，Adam，cosine学習率0.0005→0.000005とし，色・校正グループや過去入力の整合度に基づく正の行重要度を使用した．

データ拡張として，3×3入力の50%にRGB別gain 0.75〜1.30とoffsetを加えた．灰・白には30%の確率で共通gain 0.5〜1.15を適用した．offset幅は学習入力間の距離から定め，設定と行重要度をコード・CSVへ保存した．これは学習時の変換であり，物理測定の追加ではない．

5.3　実装アルゴリズムとソースコード

PCで学習・閾値選択・枝刈り・CSR変換を行い，Dense重みとCSR配列を数値ヘッダへ出力した．Arduinoでは共通入力から両方式を計算した．重みの形式は両方式ともFP32とした．

CSR変換では各層を行順に走査し，非ゼロ重みを `values`，列番号を `col`，各行の開始位置と終端を `row` へ格納した．列番号は昇順とし，加算順序を元の行列にそろえた．

`trained_models::predict()` はDenseの3層の推論を行う．`comparison_models::sparseLayer()` はCSR形式の1層を計算し，`comparison_models::predict()` は条件番号に応じてDenseまたは3層のCSR推論を呼び出す．隠れ層のみReLUを適用し，中間値を2本の作業配列へ保持する．

新規読取では `normalizeInput()` が黒白校正値による正規化とクリップを行い，`comparison_models::inferAll()` が同一入力から全8条件の出力を求める．`outputTo255()` は出力を0〜255の整数へ変換する．整数RGBをPPMとして保存し，正解画像とのMSEを算出する．

validIndices() で行位置の単調性・終端・列範囲を，selfCheckAll() で固定入力に対する出力を検査した．CSR変換の正しさは枝刈り後のDense計算と照合し，精度への影響は枝刈り前のDenseとのMSE差で評価した．

学習は train.py，候補比較は select_compression.py，追加条件による閾値固定は finalize_compression.py，ヘッダ出力は build_program.py を使用した．受領ログは record_fresh_capture.py，単独ビルド表示は record_single_model_memory.py で検算・集計した．

CSR推論と計時の関数は，付録の「コメント付きコード」に行ごとの説明を付して掲載する．

5.4 閾値選択

学習済み重みの絶対値について，隣接する異なる値の中間を候補とし，候補数は固定しなかった．検証128入力でMSE増加5%以下かつDenseより小容量となり，学習内V8の9入力でもMSE300以下となる候補から最小容量のものを選んだ．新規10回の開始前に重み・閾値を固定した．

下表は選択に使用した入力での結果である．検証2回目には異なる6色が同じ0入力になる問題があり，選択は暫定とした．

H	検証Dense MSE	検証CSR MSE	変化率	学習内V8の9入力CSR MSE
40	5012.20	4715.84	-5.91%	277.51
60	5226.80	5292.58	+1.26%	260.18
80	5135.43	4553.44	-11.33%	259.25
100	5237.21	4632.38	-11.55%	290.37

5.5 紙の新規10回読取

TEACHER_FRESH_V10 で，各回に黒b・白wを登録した．各RGBの読取値を式(1)で正規化し，入力とした．

赤・緑・青・黄・水色・紫・黒・灰・白の順に紙を当て直し，rを各1回送信した．同じ新規入力を全8モデルへ与え，9色終了後に8個のMSEと8枚のPPMを保存した．nで次回の校正へ進み，CAPTURE=1〜10を実施した．反復単位は9色の読取1回であり，内部10サンプル平均は反復回数に数えない．

RAW・黒白校正値・正規化入力・モデル識別値・閾値・PPMを全回保存した．第2回は高MSEだったが，操作ミスの申告はなく，全10回を集計した．除外した回はない．

5.6 速度と容量の測定

TEACHER_RESULT_COMPARE_V10 の timeModel() は，指定モデルの推論を保存済み実測9入力で100回行い，micros() [4]で経過時間を求める関数である．comparison_models::predict() で推論を呼び出し，出力の合計をチェックサムとして保持する．runBenchmarks() は各モデル10回のウォームアップ後に100推論×5組を計時し，Dense先行とCSR先行を交互にする．経過時間を100で割り，1推論当たりの時間を求めた．

計時区間は推論・入力選択・チェックサム加算を含み，センサ読取・シリアル出力・MSE計算を除く．新規10回の読取時間や，推論関数単体の時間とは異なる．

容量は，重み・バイアス・CSR索引の数値配列，8条件入りプログラム全体，単独モデル入りプログラム全体を分けて評価した．単独比較は memory_probe_v10 で8条件を各1回コンパイルし，Arduino IDEのFlash・静的RAM表示を記録した．共通の初期化・推論呼出し・シリアル出力を含み，センサ読取・MSE計算を除く．単独版の実行確認，IDE・コア版とバイナリは未記録である．

5.7　計画から変更した実装とその理由

学習データと損失.　読取・校正による入力変動に対応するため，33組から追加実測を含む135行へ増やし，gain・offsetによるデータ拡張を導入した．大きな残差の影響を抑えるため，MSE損失から重み付きSmooth L1へ変更した．全行を正の重要度で残し，評価にはMSEを用いた．

正解画像.　3×3の教師値と評価には `result.ppm` を使用した．

次元と配置.　Hの増加による容量・速度・精度の変化を調べるため，40・60に80・100を追加し，50は未実施とした．モデルをconst配列としたため，授業例の搭載可否をそのまま適用できない．H100のDense配列43,212 byteに対し静的RAMは6,764 byteであり，配列容量と静的RAM表示は一致しない．リンカマップは未保存である．

評価範囲.　最終測定日に追加チャートや印刷を利用できず，独立最終評価から学習チャートの新規10回読取へ変更した．同一入力での方式差と読取変動を評価するが，未知色への汎化性能は検証していない．保存入力の実機再生は物理読取と分けて記録した．

6　結果

6.1　新規10回のMSE

表1　全10回のMSE集計

H	方式	平均MSE	標本SD	最小	最大	300以下の回数
40	DENSE	319.56	903.46	5.29	2889.14	9/10
40	CSR	452.04	923.64	112.70	3078.33	9/10
60	DENSE	238.33	738.23	1.22	2339.37	9/10
60	CSR	493.51	814.80	177.18	2811.55	9/10
80	DENSE	231.68	646.48	1.25	2069.25	9/10
80	CSR	465.72	683.36	202.44	2407.85	8/10
100	DENSE	260.42	643.68	2.44	2063.62	8/10
100	CSR	651.92	694.07	318.85	2576.07	0/10

平均MSE300以下はDenseのH60・80・100であった．CSRは全Hで平均300を超え，H100は全回で300を超えた．

7　本稿で省略した内容

対応差，モデル配列容量，推論時間，Flash・RAMの結果，考察，結論および各回の実測値は，完成版に掲載している．本稿はその一部を確認するための抜粋である．使用関数の役割を本文で説明し，コメント付きコードを付録に掲載する．

8　参考文献

[1] R. Tibshirani, “Regression Shrinkage and Selection via the Lasso,” Journal of the Royal Statistical Society: Series B, 58(1), pp. 267–288, 1996.

[2] S. Han, J. Pool, J. Tran, and W. J. Dally, “Learning Both Weights and Connections for Efficient Neural Networks,” Advances in Neural Information Processing Systems 28, pp. 1135–1143, 2015.

[3] N. Bell and M. Garland, “Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors,” Proceedings of SC ’09, Article 18, pp. 1–11, 2009.

[4] Arduino, “micros(),” Arduino Documentation, https://docs.arduino.cc/language-reference/en/functions/time/micros/.

[5] PyTorch Contributors, “SmoothL1Loss,” PyTorch Documentation, https://docs.pytorch.org/docs/stable/generated/torch.nn.SmoothL1Loss.html.

9 付録：コメント付きコード

実際に使用した teacher_compare_v10.ino の関数に，行ごとの説明コメントを加えて示す.

9.1 CSR層の推論

```
void sparseLayer( // CSR形式の1層を計算する関数
  const comparison_models::SparseLayer &layer, // 層の重みと位置情報
  const float *input, // 入力配列を受け取る
  float *output, // 計算結果を書き込む配列
  bool relu // ReLUを適用するかの指定
) { // 関数の処理を開始する
  for (int r = 0; r < layer.rows; r++) { // 出力先の各行を順に計算する
    float value = layer.bias[r]; // その行のバイアスを合計の初期値にする
    for (uint16_t k = layer.row[r]; // その行の非ゼロ重みの開始位置
         k < layer.row[r+1]; // 次の行の開始位置の手前まで繰り返す
         k++) // 次の保存位置へ進む
      value += layer.values[k] * input[layer.col[k]]; // 重みと対応入力の積を加える
    output[r] = (relu && value < 0) ? 0 : value; // 指定時は負の値を0にする
  } // その行の計算を終える
} // 関数を終える
```

9.2　推論時間の計測

```
uint32_t timeModel( // 100推論の経過時間を返す関数
  int model, // 計時するモデルの番号
  int group, // 計時する組の番号
  double &checksum // 出力の合計を呼出元へ返す参照
) { // 関数の処理を開始する
  float output[3]; // 推論結果のRGBを一時保存する
  checksum = 0; // 出力の合計を初期化する
  uint32_t begin = static_cast<uint32_t>(micros()); // 開始時刻を記録する
  for (int call = 0; call < 100; call++) { // 100回の推論を繰り返す
    int input = (group * 100 + call) // 組を通した呼出番号を求める
      % comparison_models::BENCHMARK_INPUT_COUNT; // 9入力を巡回する番号にする
    comparison_models::predict( // 指定モデルの推論を呼び出す
      model, // モデル番号を渡す
      comparison_models::BENCHMARK_INPUTS[input], // 選んだ保存入力を渡す
      output); // 結果をoutputに受け取る
    checksum += output[0] + output[1] + output[2]; // RGBの出力を合計へ加える
  } // 100回の繰り返しを終える
  return static_cast<uint32_t>(micros()) - begin; // 経過時間をμsで返す
} // 関数を終える
```